

大型語言模型與資訊安全系統

Applying Large Language Models in Cybersecurity Systems

Report 2：ThreatRadar 資安新聞即時威脅雷達

組別	第 74 組
組名	春風不解風情
組員姓名	鍾唐福
學號	M11415121
負責模組	C：Dashboard / UX
Date	2026/05/25

本階段摘要：第二週的重點是把 Report 1 做好的 Dashboard 原型，接到更完整的模型分析流程。本週我確同一個 LLM 分析步驟已經可以透過設定切換到線上 API，並以 DeepSeek 作為 primary provider 完成多輪 crawl + analyze。後來重新執行時，也確認本地 llama3.2 已載入，因此目前系統不只可以展示雲端模型分析結果，也具備後續做雙模型比較的條件。整體流程包含新聞與 CVE 收集、LLM 分析、SQLite 儲存、Dashboard 顯示，以及匯出報表。

相較於第一份進度報告，這次比較像是把各模組串起來做一次完整檢查。除了畫面本身，我也特別確認資料從 crawler 進到 database、再由 provider 分析、最後回到 Dashboard 的路徑是否順利。

1. Report 1 到 Report 2 的進度差異

Report 1 時，我的工作比較集中在 C 模組 Dashboard / UX：例如搜尋、排序、掃描歷史、重跑分析與系統狀態頁。Report 2 的重點則往前推了一步，不只是介面能顯示資料，而是要確認資料背後的模型分析流程真的可以切換與執行。

在模型來源上，原本系統主要依賴本地 Ollama llama3.2。這週加入 provider abstraction 後，PRIMARY_PROVIDER 可以在 local 與 cloud 之間切換。這個改動的好處是 Dashboard 與資料庫結構不需要跟著大改，後續只要調整環境變數就能選擇主要分析模型。

線上 API 也從原本的規劃變成可以實跑的功能。本週新增 CloudProvider，透過 OpenAI-compatible /chat/completions 介面呼叫 DeepSeek API。從終端機紀錄可以看到 50/50、22/22、6/6、2/2 items 都完成分析，代表 API 已經接進原本的 pipeline。

雙模型展示的條件也比上一週完整。最新系統狀態顯示本地 llama3.2 已載入，雲端 DeepSeek 也已設定。目前 ANALYSIS_MODE 仍維持 single，主要結果由 DeepSeek 寫入 news 表；但接下來若要展示 compare 或 hybrid 模式，基礎環境已經準備好。

目前比較需要注意的是 CISA RSS feed 會回傳 403。這不是整個 pipeline 失敗，因為其他 RSS 與 NVD 仍能正常新增資料；比較像是單一資料來源的存取限制，demo 時需要先說明清楚。

因此第二週的風險點也跟第一週不同。第一週主要看 UI 是否能清楚呈現資料；這週更需要確認模型來源切換、API key 設定、分析結果寫回，以及 Dashboard 是否能正確顯示目前實際使用的模型。

2. 更新後的系統 Pipeline

目前 main.py 仍是唯一入口。程式啟動後會初始化 SQLite、執行一次 crawl + analyze、啟動 APScheduler，最後 launch Gradio。這週的主要差異在 analyzer 層：它不再直接綁死 Ollama，而是先根據 config 選擇 provider，因此同一套分析流程可以切換成本地模型或線上 API。

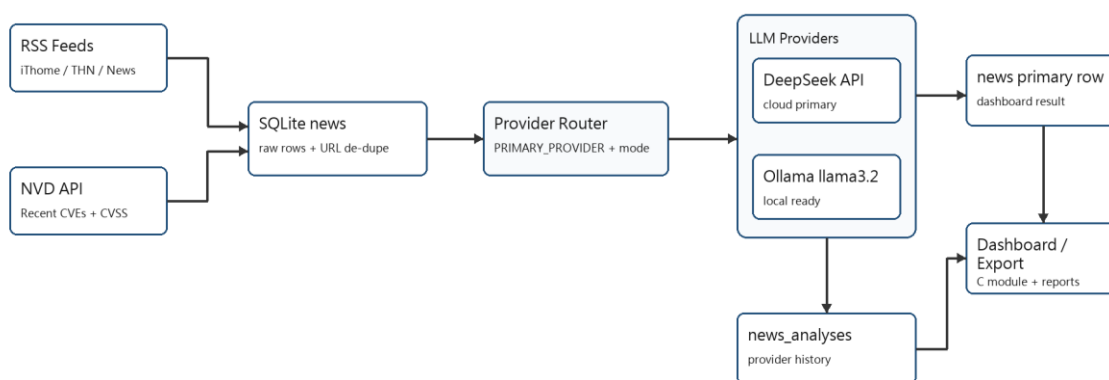


圖 1：ThreatRadar 更新後 Pipeline 架構（雙模型 Provider Router）

資料流大致可分成兩條。第一條是主要威脅情報路徑：RSS 與 NVD 先寫入 news 表，Provider Router 依照 PRIMARY_PROVIDER 呼叫主要模型，最後把威脅等級、CVE、受影響產品與行動建議寫回 news，供 Dashboard 直接讀取。第二條是模型歷史與比較路徑：每次 provider 的輸出會另外保存到 news_analyses，之後可用來比較 DeepSeek 與 llama3.2 對同一篇新聞的判斷差異。

這樣的分層對期末展示很有幫助。目前 demo 先用 single 模式讓 DeepSeek 寫入主要結果；同時，系統狀態頁已確認本地 llama3.2 可以連上。換句話說，雙模型比較不是臨時加上的口號，而是可以在下一步透過 ANALYSIS_MODE=compare 實際展示。

從 C 模組角度看，Dashboard 不只是把資料列出來而已，它也是整個 pipeline 的觀察介面。新聞卡片可以讓使用者確認分析結果是否已寫回；系統狀態頁則可以看出待分析筆數、失敗筆數與最近爬取時間。這些資訊能讓後端流程變成可檢查、可展示的證據。

3. LLM Provider 切換與 DeepSeek API 實作

本週主要技術改動是把 LLM 呼叫抽象成 provider。config.py 新增 PRIMARY_PROVIDER、ANALYSIS_MODE、CLOUD_LLM_PROVIDER、CLOUD_LLM_BASE_URL、CLOUD_LLM_MODEL 與

CLOUD_LLM_API_KEY。這樣使用者不需要直接改程式碼，只要調整環境變數，就能把主要分析模型從本地 Ollama 切到 DeepSeek、Qwen 或其他 OpenAI-compatible API。

CloudProvider 採用 OpenAI-compatible 的 /chat/completions schema，因此 DeepSeek、Qwen、OpenAI 或其他 gateway 都可以在相同介面下使用。本次實測設定 CLOUD_LLM_PROVIDER=deepseek、CLOUD_LLM_MODEL=deepseek-v4-flash，並由 DeepSeek 作為 primary provider 寫入 news 表。

這個 abstraction 的價值在於降低切換模型的成本。如果 Ollama 呼叫直接寫死在 analyzer/llm.py，之後要換 DeepSeek 就可能牽動 prompt、JSON parsing、retry 與資料庫更新。現在本地模型與雲端模型都回傳相同 schema，Dashboard 只需要讀取 news 表中的主要結果，不必知道背後是哪一個模型服務。

另外，我也保留相同的 prompt 與輸出格式。無論模型是本地 llama3.2 或 DeepSeek API，分析結果都會整理成 threat_level、cve_ids、affected_products 與 action_summary。這點對本專題很重要，因為 Dashboard 與 GitHub Scanner 後續都會依賴這些欄位；如果不同模型輸出格式差太多，前端呈現與 repo dependency matching 都會變得不穩定。

本次設定方式：本次實測使用 PowerShell 設定 PRIMARY_PROVIDER=cloud、ANALYSIS_MODE=single、CLOUD_LLM_PROVIDER=deepseek、CLOUD_LLM_BASE_URL=https://api.deepseek.com、CLOUD_LLM_MODEL=deepseek-v4-flash。API key 只放在環境變數中，不寫入 GitHub，避免把憑證放進專案。

```
23 # -- Local LLM (Ollama) -----
24 # Override via env var so Docker can point to the ollama service.
25 OLLAMA_BASE_URL = os.environ.get("OLLAMA_BASE_URL", "http://localhost:11434")
26 OLLAMA_MODEL = os.environ.get("OLLAMA_MODEL", "llama3.2")
27 OLLAMA_TIMEOUT = 120
28
29 # Stable identifier for the local provider's results in news_analyses.
30 LOCAL_PROVIDER = "ollama"
31
32 # -- Cloud LLM (second provider, optional) -----
33 # Kept vendor-neutral: cloud_provider.py speaks the OpenAI-compatible
34 # /chat/completions schema, which OpenAI, Together, Groq, OpenRouter, vLLM, etc.
35 # all expose. Leave CLOUD_LLM_API_KEY empty to disable the cloud provider - the
36 # adapter then reports status="skipped" instead of erroring.
37 CLOUD_LLM_PROVIDER = os.environ.get("CLOUD_LLM_PROVIDER", "openai-compatible").strip()
38 CLOUD_LLM_MODEL = os.environ.get("CLOUD_LLM_MODEL", "").strip()
39 CLOUD_LLM_API_KEY = os.environ.get("CLOUD_LLM_API_KEY", "").strip()
40 CLOUD_LLM_BASE_URL = os.environ.get("CLOUD_LLM_BASE_URL", "https://api.openai.com/v1").strip()
41 CLOUD_LLM_TIMEOUT = int(os.environ.get("CLOUD_LLM_TIMEOUT", "120"))
42
43 # Which model is AUTHORITATIVE - It fills the news table (the dashboard's primary
44 # view) and drives the retry budget. Switch the whole pipeline between the local
45 # model and an online API by changing this one value:
46 # local -> Ollama (config below)
47 # cloud -> the online API configured in the CLOUD_LLM_* block (DeepSeek, Qwen,
48 # OpenAI, Groq, ... any OpenAI-compatible endpoint)
49 PRIMARY_PROVIDER = os.environ.get("PRIMARY_PROVIDER", "local").strip().lower()
50 if PRIMARY_PROVIDER in ("ollama", "local"):
51     PRIMARY_PROVIDER = "local"
52 elif PRIMARY_PROVIDER in ("cloud", "api", "online"):
53     PRIMARY_PROVIDER = "cloud"
54 else:
55     PRIMARY_PROVIDER = "local"
56
57 # Analysis mode controls whether a SECOND model also runs (for comparison):
58 # single - only the primary model (default).
59 # compare - run primary + the other model on every item, store both.
60 # hybrid - primary on every item; the other only re-checks HIGH/CRITICAL items.
61 # "local_only" is accepted as a legacy alias for "single".
62 ANALYSIS_MODE = os.environ.get("ANALYSIS_MODE", "single").strip().lower()
63 if ANALYSIS_MODE in ("local_only", "primary_only", "only"):
64     ANALYSIS_MODE = "single"
```

程式碼佐證 A：config.py 26-64：Provider 與分析模式設定

```

27 class AnalysisProvider(abc.ABC):
28     provider: str = "unknown"
29     model: str = "unknown"
30
31     @abc.abstractmethod
32     def _call(self, prompt: str) -> str:
33         """Send the prompt to the model and return its raw text response (JSON)."""
34         raise NotImplementedError
35
36     def _base_result(self, status: str, **extra) -> dict:
37         base = {
38             "provider": self.provider,
39             "model": self.model,
40             "prompt_version": PROMPT_VERSION,
41             "status": status,
42             "threat_level": None,
43             "cve_ids": [],
44             "affected_products": [],
45             "action_summary": "",
46             "latency_ms": None,
47             "error": None,
48             "warnings": [],
49         }
50         base.update(extra)
51         return base
52
53     def analyze(self, title: str, content: str) -> dict:
54         """Run the full prompt -> call -> parse -> normalize pipeline, timed."""
55         prompt = build_prompt(title, content)
56         start = time.monotonic()
57         try:
58             raw = self._call(prompt)
59             latency = int((time.monotonic() - start) * 1000)
60             parsed = json.loads(raw)
61             norm = normalize_analysis_result(parsed)
62             return self._base_result(
63                 "ok",
64                 threat_level=norm["threat_level"],
65                 cve_ids=norm["cve_ids"],
66                 affected_products=norm["affected_products"],
67                 action_summary=norm["action_summary"],
68                 latency_ms=latency,
69                 warnings=norm["warnings"],
70             )
71         except Exception as e: # network error, bad JSON, etc.
72             latency = int((time.monotonic() - start) * 1000)
73             return self._base_result("error", latency_ms=latency, error=f"{type(e).__name__}: {e}")

```

程式碼佐證 B：analyzer/providers.py 27-58：共同 Provider 介面與輸出格式

```

20 class CloudProvider(AnalysisProvider):
21     def __init__(self):
22         self.provider = config.CLOUD_LLM_PROVIDER or "cloud"
23         self.model = config.CLOUD_LLM_MODEL or "unconfigured"
24
25     def analyze(self, title: str, content: str) -> dict:
26         # Guard before doing any work so an unconfigured cloud never errors.
27         if not config.cloud_enabled():
28             return self._base_result(
29                 "skipped",
30                 error="cloud LLM not configured (set CLOUD_LLM_API_KEY and CLOUD_LLM_MODEL)",
31             )
32         return super().analyze(title, content)
33
34     def _call(self, prompt: str) -> str:
35         resp = requests.post(
36             f"{config.CLOUD_LLM_BASE_URL.rstrip('/')}/chat/completions",
37             headers={
38                 "Authorization": f"Bearer {config.CLOUD_LLM_API_KEY}",
39                 "Content-Type": "application/json",
40             },
41             json={
42                 "model": config.CLOUD_LLM_MODEL,
43                 "messages": [{"role": "user", "content": prompt}],
44                 "temperature": 0.1,
45                 # Ask for JSON object output where the provider supports it.
46                 "response_format": {"type": "json_object"},
47             },
48             timeout=config.CLOUD_LLM_TIMEOUT,
49         )
50         resp.raise_for_status()
51         return resp.json()[0]["message"]["content"]
52

```

程式碼佐證 C：analyzer/cloud_provider.py 20-51：DeepSeek / OpenAI-compatible API 呼叫

```

170 def analyze_single(news_id: int, title: str, content: str) -> bool:
171     from analyzer.providers import make_provider # lazy import breaks import cycle
172
173     mode = config.ANALYSIS_MODE
174
175     # 1) Primary provider (local OR cloud, per PRIMARY_PROVIDER).
176     primary_kind = config.primary_provider_kind()
177     primary = make_provider(primary_kind).analyze(title, content)
178     _persist_provider_result(news_id, primary)
179
180     primary_ok = primary.get("status") == "ok"
181     if primary_ok:
182         update_analysis(
183             news_id=news_id,
184             threat_level=primary["threat_level"],
185             cve_ids=json.dumps(primary["cve_ids"], ensure_ascii=False),
186             affected_products=json.dumps(primary["affected_products"], ensure_ascii=False),
187             action_summary=primary["action_summary"],
188         )
189     else:
190         retries = mark_analysis_failed(news_id, MAX_ANALYSIS_RETRIES)
191         tag = "giving up" if retries >= MAX_ANALYSIS_RETRIES else "will retry"
192         print(f"[LLM] news {news_id} primary[{primary_kind}] {tag} "
193               f"({retries}/{MAX_ANALYSIS_RETRIES}): {primary.get('error')}")
194
195     # 2) Secondary provider, per mode.
196     if _should_run_secondary(mode, primary):
197         secondary_kind = config.secondary_provider_kind()
198         sres = make_provider(secondary_kind).analyze(title, content)
199         _persist_provider_result(news_id, sres)
200         if sres.get("status") != "ok":
201             print(f"[LLM] news {news_id} secondary[{secondary_kind}] "
202                   f"status={sres.get('status')}: {sres.get('error')}")
203
204     return primary_ok
205
206
207 def analyze_pending_news() -> int:
208     pending = get_unanalyzed_news(limit=50)
209     if not pending:
210         return 0
211
212     # Misconfiguration guard: cloud chosen as primary but no key/model set.
213     if config.primary_provider_kind() == "cloud" and not config.cloud_enabled():
214         print("[LLM] WARNING: PRIMARY_PROVIDER=cloud but CLOUD_LLM_API_KEY/"
215               "CLOUD_LLM_MODEL are not set - every item will be marked 分析失敗.")
216
217     success = 0
218     with ThreadPoolExecutor(max_workers=LLM_CONCURRENCY) as pool:
219         futures = [
220             pool.submit(analyze_single, item["id"], item["title"], item.get("raw_content", ""))
221             for item in pending
222         ]
223         for f in futures:
224             if f.result():
225                 success += 1
226
227     prov, model = config.primary_label()
228     print(f"[LLM] Analyzed {success}/{len(pending)} items "
229           f"(primary={prov}/{model}, mode={config.ANALYSIS_MODE}, concurrency={LLM_CONCURRENCY})")
230     return success

```

程式碼佐證 D：analyzer/llm.py 170-214：primary / secondary 模型分析流程

4. 實跑結果與 Dashboard 證據

我用 .venv 內的 python main.py 重新執行後，系統可以初始化資料庫、執行排程爬取、呼叫 DeepSeek 分析，並在 <http://127.0.0.1:7860> 開啟 Dashboard。終端機 log 顯示 primary=deepseek/deepseek-v4-flash，表示 cloud API 已接進原本的分析流程；系統狀態頁也顯示本地 llama3.2 已載入，代表 local provider 的前置條件也補齊了。

```

Terminal execution evidence -- DeepSeek API pipeline

.\venv\Scripts\python.exe main.py
[Main] Database initialized
[Scheduler] Starting crawl cycle...
[RSS] Error crawling CISA Alerts: 403 Client Error: Forbidden ...
[LLM] Analyzed 50/50 items (primary=deepseek/deepseek-v4-flash, mode=single, concurrency=3)
[Scheduler] 爬取完成：RSS +0 筆，NVD +0 筆，分析 50 筆
[Main] Scheduler started (every 60 min)
[Main] Launching dashboard at http://localhost:7860
Running on local URL: http://0.0.0.0:7860
後續排程：22/22、6/6、2/2 items 均由 DeepSeek 完成分析

```

圖 2：main.py 實跑紀錄：DeepSeek API 完成 LLM 分析

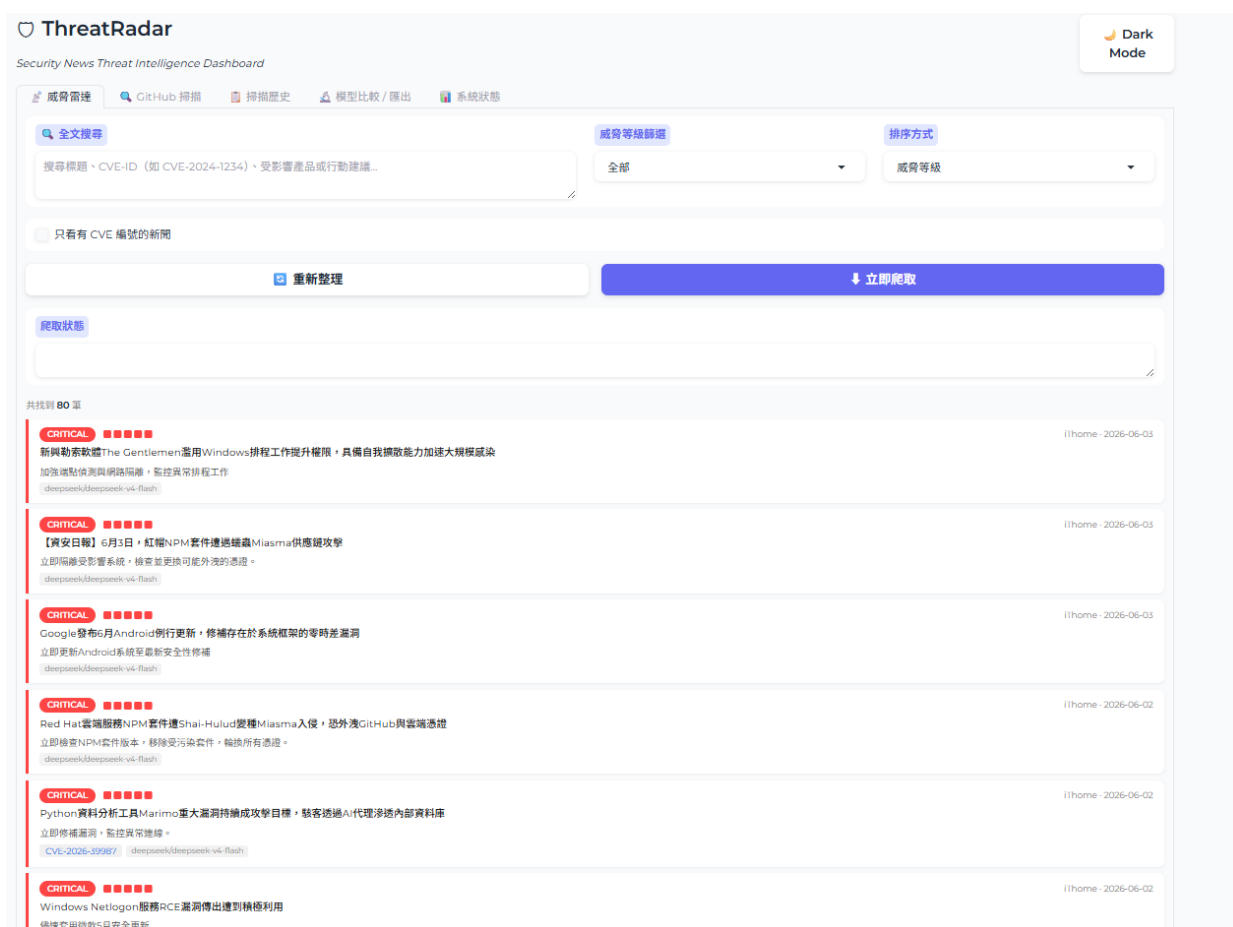


圖 3：威脅雷達頁：DeepSeek 分析結果與模型來源標籤

從圖 3 可以看到，威脅卡片會顯示 CRITICAL 等級、摘要建議、CVE chip，以及 deepseek/deepseek-v4-flash 的模型標籤。這可作為 Dashboard 已讀到 DeepSeek 分析結果的佐證，而不是只顯示舊資料。

威脅雷達頁也比較符合這個專題想解決的問題：資安人員不一定有時間逐篇閱讀完整新聞，因此畫面需要先把事件急迫性、CVE、受影響產品與建議行動整理出來。這比單純列出 RSS 標題更接近威脅情報 dashboard 的使用情境。

5. 系統狀態、資料統計與輸出設計

系統狀態頁主要用來確認 PoC 是否真的在運作，而不是只有靜態畫面。最新截圖顯示已分析新聞 261 筆、待分析 0 筆、分析失敗 0 筆；同時，本地模型 llama3.2 已載入，cloud provider 也設定為 deepseek/deepseek-v4-flash。這代表目前資料沒有堆積，兩種模型來源也都具備後續比較的條件。

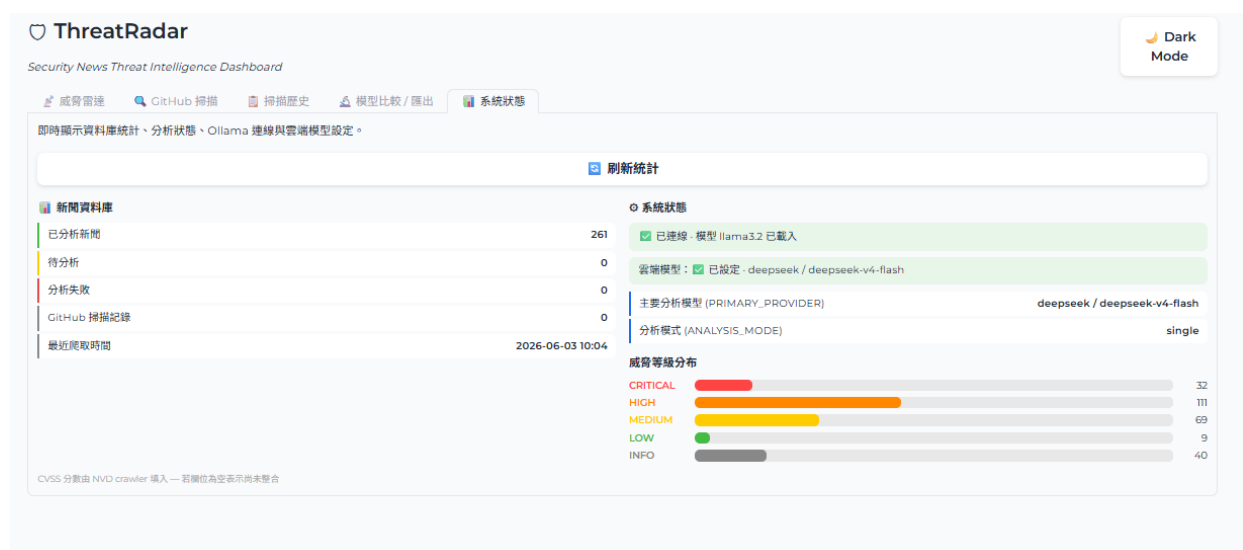


圖 4：系統狀態頁：llama3.2 已載入與 DeepSeek primary provider 設定

觀察結果：這次重新執行後，Ollama 連線檢查已顯示 llama3.2 載入成功，雲端 DeepSeek 也完成 primary provider 設定。因此這份報告可以同時說明線上 API 已跑通，以及本地模型已準備好進入 compare 模式。

這個狀態頁在 demo 時很實用。一方面，它能让評審直接看到目前分析模型與資料庫統計，不需要只靠口頭說明；另一方面，它也能用來除錯，例如確認待分析筆數是否累積、失敗筆數是否增加、最近爬取時間是否更新。對 C 模組而言，這是把後端狀態轉成可理解資訊的關鍵畫面。

從最新統計來看，已分析 261 筆、待分析 0 筆、失敗 0 筆，表示資料庫內目前沒有堆積未處理項目。威脅等級分布中 HIGH 與 MEDIUM 佔多數，也有一定數量的 CRITICAL 事件，因此 Dashboard 的排序與篩選功能有實際展示價值，不會只是空白畫面或單一等級資料。

6. Provider History、模型比較與匯出

為了支援雙模型設計，本週新增 news_analyses 表。news 表仍保留 Dashboard 預設顯示的主要結果，而 news_analyses 以 append-only 方式保存每次 provider 的完整輸出，包含 provider、model、prompt_version、threat_level、cve_ids、affected_products、latency_ms、status 與 error。

匯出功能補上了報告與 demo 會需要的 output 檔案。exporter/report_exporter.py 可以輸出 threat_report CSV、threat_report JSONL，以及 model_comparison CSV。CSV 使用 utf-8-sig，方便 Excel 正確顯示繁體中文；JSONL 則保留 list 欄位，方便後續評測或人工檢查。

目前輸出檔案分成三種。threat_report_*.csv 適合放進報告或用 Excel 檢查，欄位包含新聞來源、CVE、產品、威脅等級、行動建議與模型資訊。threat_report_*.jsonl 保留 cve_ids 與 affected_products 的 array 型態，未來可以接 evaluation dataset 或人工標註流程。model_comparison_*.csv 則是雙模型模式的展示重點，可以把 local 與 cloud 的 threat_level 差異整理成可討論的資料。

這個 output 設計讓成果不只停留在 Gradio 介面。CSV 可以放進進度報告、期末簡報或人工驗證流程；JSONL 可以作為後續 evaluation dataset 的雛形；model_comparison.csv 則可以把雙模型差異轉成可量化資料，例如觀察兩個模型在 CRITICAL/HIGH 判斷上的一致率，或找出 action_summary 明顯不同的案例。

7. 待加強項目與下週計畫

目前整體程式已經可以跑通：DeepSeek API 能完成 LLM 分析，Dashboard 能顯示結果，系統狀態也能反映資料庫與 provider 設定。不過如果要讓 final demo 更穩定，我認為還有幾個地方可以補強。

首先是 CISA RSS 403。這不是整體 pipeline 的錯誤，因為其他 RSS 與 NVD 仍能正常新增資料；但 demo 時如果看到錯誤訊息，容易被誤解成爬蟲失敗。後續可以在 requests headers 加入 User-Agent，或在簡報中說明這是單一來源的存取限制。

第二個是 cloud primary 未設定時的保護。目前如果 PRIMARY_PROVIDER=cloud 但忘記設定 API key 或 model，系統可能會把資料列入 retry。比較好的做法是在 analyze_pending_news() 一開始就檢查設定；若 cloud primary 未啟用，就直接跳過，不消耗 retry budget。

第三個是 API 相容性。雖然 DeepSeek 已經跑通，但不同 OpenAI-compatible provider 對 response_format 的支援程度不一定相同。如果後續要強調可切換 Qwen、OpenRouter 或其他 gateway，CloudProvider 可以在 response_format 不支援時自動 fallback 成一般 JSON prompt 呼叫。

第四個是雙模型展示。目前 single 模式已足以說明 DeepSeek pipeline 可以運作，而且本地 llama3.2 也已載入；之後若要更符合「兩個 LLM 分析」的專題亮點，可以將 ANALYSIS_MODE 改成 compare，讓模型比較頁與 model_comparison.csv 產出更完整的證據。

最後是 GitHub Scanner 的展示資料。目前系統已把 affected_products 作為新聞分析與 repo 掃描的銜接欄位，但 demo 時最好準備一個包含 requirements.txt 或 package.json 的測試 repository。這樣才能實際展示「新聞威脅情報」如何轉成「我的專案是否可能受影響」的個人化檢查結果。

Report 2 結論：第二週的主要成果，是把 ThreatRadar 從本地 LLM PoC 推進到可切換線上 API 的版本。目前系統可以用 DeepSeek 作為主要分析模型，自動爬取資安新聞與 NVD CVE，將分析結果寫入 SQLite，並在 Dashboard 顯示威脅等級、行動建議、CVE 與模型來源。對我負責的 C 模組來說，這週的重點不只是美化畫面，而是讓 Dashboard 成為可以驗證整體 pipeline 的操作介面，後續可用來支援期末 demo 與最終報告。